



The Open Source Audit

Subject: VLC Media Player

Capstone Project | Units 1-5 | Max Marks: 100

Student Name	Vani Varada
Registration No.	24MIP10023
Slot	A24
Date of Submission	31 March 2026
Software Chosen	VLC Media Player (LGPL v2.1 / GPL v2)

Table of Contents

1. Introduction	3
2. Part A — Origin and Philosophy	3
A1. How VLC Was Born at a French University	3
A2. The LGPL and GPL Licenses Explained	5
A3. Ethics, Codecs, and the Open Source Debate	6
3. Part B — VLC's Footprint on Ubuntu Linux	7
4. Part C — VLC in the FOSS Ecosystem	8
5. Part D — Open Source vs Proprietary: Critical Analysis	9
6. Shell Scripts Documentation	10
Script 1 — System Identity Report	10
Script 2 — FOSS Package Inspector	11
Script 3 — Disk and Permission Auditor	12
Script 4 — Log File Analyzer	13
Script 5 — Open Source Manifesto Generator	14
7. References	15

1. Introduction

There is a small orange traffic cone on millions of desktops around the world. It is the icon of VLC Media Player — the free and open source multimedia player developed by VideoLAN that plays almost any audio or video file you throw at it, regardless of format, codec, or DRM restriction. It runs on Windows, macOS, Linux, Android, iOS, and a handful of other platforms. It has been downloaded over five billion times. And it was created by students.

I chose VLC for this audit because it is the kind of software story that does not get told enough. Unlike the Linux kernel or Git, which came from established developers solving well-defined technical problems, VLC came from a group of students at École Centrale Paris who needed to watch satellite TV in their university dormitory and could not find software that worked. They built their own. They open-sourced it. Twenty-five years later it is one of the most widely used media players on earth, maintained by a non-profit organisation with contributors in dozens of countries.

VLC is developed by VideoLAN and licensed under the GNU Lesser General Public License version 2.1 (LGPL v2.1), with some components under GPL v2. This dual-license structure is unusual and worth understanding — it reflects a deliberate philosophy about how media software should relate to the rest of the software ecosystem. This report examines that philosophy, VLC's technical footprint on Linux, the ecosystem it has built, and a comparison with the proprietary alternatives it has largely made irrelevant.

2. Part A — Origin and Philosophy

A1. How VLC Was Born at a French University

École Centrale Paris, 1996

The story of VLC begins not in a corporate lab or a hacker's bedroom, but in the network operations room of a French engineering school. In 1996, students at École Centrale Paris were managing the university's campus network and wanted to distribute a satellite TV feed to dormitory rooms across the campus. The idea was simple: receive a satellite signal, encode it digitally, and stream it over the university's local network so students could watch television on their computers without everyone needing their own satellite dish.

The problem was that no existing software could do what they needed. The media players of the mid-1990s were almost entirely proprietary, tightly tied to specific operating systems, and completely incapable of handling network-streamed multimedia. Windows Media Player existed but was Windows-only and could not receive networked streams. QuickTime was macOS-only. RealPlayer was a proprietary, ad-laden mess. Nothing was cross-platform, nothing handled raw streams, and nothing was modifiable.

So the students built their own. The project was called VideoLAN — a Video Local Area Network — and the client component was VLC: VideoLAN Client. The first version was a closed internal tool, never meant to leave the university. It ran only on Linux, did not have a graphical interface, and was controlled entirely from the command line. Its only purpose was to receive the MPEG-2 satellite feed and display it on screen. But it worked, and that was the beginning.

The Decision to Open Source VLC

In 2001, the VideoLAN team made the decision that would define VLC's future: they released the source code under the GNU General Public License version 2. The motivation was both practical and philosophical. Practically, the team was graduating and leaving École Centrale — the project would die unless it could be maintained by a broader community. Philosophically, the students who built VideoLAN had grown up on free software and believed their work belonged to the community that had given them the tools to build it.

The release of the source code immediately attracted contributors. Developers added support for new codecs, new operating systems, and new streaming protocols. The graphical interface was developed. DVD playback was added. Within a few years, VLC had become something far more capable than the university project it started as — a universal media player that could handle practically any audio or video format, on any platform, for free.

In 2008, the VideoLAN project re-licensed VLC's core library, libvlc, under LGPL v2.1. This was a strategic decision: the LGPL allows developers to use libvlc in proprietary applications without being required to open source their entire application, which made it possible for commercial software to embed VLC's capabilities. The VLC application itself remains GPL, but the library that powers it can be used more freely. This dual-license structure reflects a mature understanding of how open source components can have the broadest possible impact.

The Cone

The orange traffic cone icon has its own story. When the VideoLAN team needed a default icon, one of the developers brought a miniature traffic cone from a model railway set and placed it on his desk. It became the project mascot, then the official icon. Today the cone is one of the most recognisable software icons in the world. It is a reminder that VLC was always slightly irreverent, built by people who were solving a real problem they had, not following a corporate roadmap.

The VideoLAN organisation today is a French non-profit association (loi 1901). Jean-Baptiste Kempf, who joined the project as a student in 2005, has been the president since 2009. VLC is maintained by a global community of volunteers and is funded through donations and corporate partnerships. It has

never shown advertisements, never collected user data, and has consistently refused to add features that would compromise the privacy or autonomy of its users.

What Problem VLC Actually Solved

VLC solved the codec problem. In the early 2000s, playing a video file on a computer was a frustrating experience. Different formats required different software. MPEG-2 needed one player, DivX needed another, and RealVideo needed RealPlayer — which also happened to install adware. DVD playback required a licensed decoder that you had to pay for separately. If you received a file in an unusual format, you spent twenty minutes hunting for the right codec pack, which often came bundled with spyware.

VLC bundled all of its codecs internally and added new ones through the libavcodec library (part of FFmpeg). The result was a player that simply worked, for everything, without asking you to install anything extra or pay anything extra. That single design decision — bundle all codecs, accept all formats, demand nothing from the user — is the philosophical heart of VLC. Media files are a form of human expression and communication. The tools to access that expression should be free, complete, and unconditional.

A2. The LGPL and GPL Licenses Explained

The Four Freedoms Applied to VLC

Freedom 0 — use for any purpose. VLC can be installed and used freely on any device for any purpose. There are no per-use fees, no regional restrictions, and no limitations on what content you play. You can use VLC in a cinema, a hospital, a school, or your bedroom — the license says nothing about purpose.

Freedom 1 — study and modify. VLC's complete source code is available at code.videolan.org (GitLab). Every line of code that makes VLC play video is publicly readable. Any developer can understand how codec demuxing works, how the audio rendering pipeline is structured, or how the network streaming module handles buffering.

Freedom 2 — redistribute copies. VLC can be freely redistributed. Linux distributions include it in their package repositories. It can be bundled in software collections. The only requirement is that the GPL terms are preserved with the distribution.

Freedom 3 — distribute modified versions. Modified versions of VLC can be distributed under GPL v2. The application code is copyleft — modifications to VLC itself must be shared. However, the libvlc library is LGPL v2.1, which means proprietary applications can embed it without being required to open source their own code, as long as they allow users to swap out the libvlc library itself.

LGPL vs GPL — Why VLC Uses Both

The distinction between LGPL and GPL is subtle but significant. The GPL is a strong copyleft license: any software that links to GPL code and is distributed must itself be GPL. The LGPL is a weaker copyleft: software can link to LGPL libraries without being required to adopt the LGPL itself. The "L" originally stood for "Library" — it was designed for software libraries that should be freely usable from both free and proprietary software.

VLC's dual licensing is strategically clever. The application — what you run when you open VLC — is GPL v2. If you fork VLC and distribute your fork, your fork must also be GPL. But the underlying libvlc library, which contains the actual multimedia decoding and playback capabilities, is LGPL. This means a mobile app developer can embed libvlc in a commercial iOS or Android application without making their entire app open source. This is why VLC's technology is embedded in hundreds of commercial products — smart TV firmware, automotive infotainment systems, streaming set-top boxes — without those products needing to be open source.

This structure maximises impact: VLC's capabilities spread as widely as possible through the LGPL library, while the reference application that the VideoLAN community develops and maintains stays

fully open source under the GPL.

Free as in Beer vs Free as in Freedom

VLC is free as in beer — it costs nothing and there is no paid tier, no premium version, and no subscription. But it is also free as in freedom in a very direct way that most software users can actually feel. When a proprietary media player refuses to play a file because you do not have the right license, or forces you to watch an advertisement before playback, or reports what you are watching to a server somewhere, you experience the loss of freedom concretely. VLC does none of these things. It plays the file. It asks nothing of you in return. That is what freedom as in freedom feels like in practice — not an abstract philosophical position but a direct, daily experience of software that respects you as a user.

A3. Ethics, Codecs, and the Open Source Debate

Should All Software Be Open Source?

VLC's history gives a specific angle on this general question. The multimedia software landscape before VLC was dominated by proprietary players that monetised user attention through advertisements, collected usage data, bundled spyware with their installers, and charged for basic functionality like DVD playback. The users had no choice but to accept these terms because there were no functional alternatives. VLC changed that by providing an alternative that was not just technically superior but philosophically opposed to those practices.

This suggests that the most important area for open source is not necessarily every category of software, but specifically those categories where proprietary dominance has led to exploitation of users. When a market is controlled by proprietary vendors who use that control to extract value from users rather than deliver it, open source alternatives serve a genuinely protective function. VLC did not just compete with Windows Media Player — it made a large part of the proprietary media player market irrelevant. That is a form of advocacy as much as it is engineering.

There are categories of software — specialised professional tools, enterprise systems with complex support requirements, security-critical components under active development — where proprietary development remains legitimate. But consumer-facing software that interacts with everyday media, communication, and information deserves to be open, because the alternative is users being held hostage to whatever the vendor decides to monetise next.

Is It Ethical to Build Commercial Products on VLC?

The LGPL license makes this question very concrete for VLC. Dozens of commercial products embed libvlc — automotive infotainment systems, smart TV platforms, commercial video editing tools. The license explicitly permits this without requiring those products to be open source. Is that ethical?

The VideoLAN team has been thoughtful about this. They chose LGPL deliberately to enable commercial adoption, because broad adoption advances their mission: a world where free multimedia software is so embedded in everything that proprietary media players become irrelevant. A Samsung smart TV that uses libvlc to play video files is a world where that TV's owner can access their media without being dependent on Samsung's proprietary codec deals. That outcome is worth enabling, even if Samsung does not contribute back to VideoLAN directly.

The ethical floor, however, is compliance with the license terms. LGPL requires that the user be able to swap out the libvlc library for a different version — a requirement that ensures the user retains some control even in proprietary products. Companies that comply with this requirement are acting within the ethical framework the license establishes. Companies that embed libvlc while preventing license compliance — using hardware or software locks to prevent library substitution — are acting unethically, and VideoLAN has pursued enforcement actions in such cases.

Open Source and Innovation in Multimedia

The history of codec and multimedia format development is one of the clearest illustrations of how open source drives innovation while proprietary approaches create fragmentation. In the 2000s and 2010s,

the multimedia landscape was a battlefield of proprietary formats: MPEG-2, H.264, AAC — all patent-encumbered, all requiring license fees from hardware manufacturers and software developers, all creating barriers to access. The result was a fragmented ecosystem where different devices could play different files, users were confused, and a significant portion of the cost of any multimedia device went to patent licensing fees.

The open source response was FFmpeg — a project that implemented free, open implementations of these codecs — and later the development of genuinely open formats: WebM, VP9, AV1, Opus. VLC was built on FFmpeg and has championed these open formats. Today, AV1 is becoming the dominant streaming codec precisely because it is royalty-free and open, and the quality-to-size ratio it achieves has made proprietary alternatives economically indefensible. That outcome is the direct result of open source development demonstrating that free can be better.

3. Part B — VLC's Footprint on Ubuntu Linux

Installation on Ubuntu

VLC is available directly from Ubuntu's universe repository and can also be installed as a Snap package from the Snap Store. Both methods are fully supported:

```
sudo apt update && sudo apt install vlc      # Install from apt (universe repo)
vlc --version                               # Confirm installed version
snap install vlc                             # Alternative: install as Snap
vlc /path/to/file.mp4                       # Play a file from terminal
vlc --intf ncurses /path/to/file.mp4        # Play with text-mode interface
```

Key Directories and Files

Path	Contents	Why It Matters
<code>/usr/bin/vlc</code>	Main VLC executable	Entry point; check with <code>which vlc</code> and <code>vlc --version</code>
<code>/usr/lib/vlc/</code>	VLC plugins (.so files) — decoders, interfaces, outputs	Each codec and output module is a loadable plugin; modular architecture
<code>/usr/lib/vlc/plugins/</code>	200+ plugin .so files organised by category	<code>codec/</code> , <code>demux/</code> , <code>access/</code> , <code>video_output/</code> , <code>audio_output/</code> sub-directories
<code>/usr/share/vlc/</code>	Skins, icons, lua scripts, HTTP interface files	VLC's web interface and Lua scripting engine live here
<code>/etc/vlc/</code>	System-wide VLC configuration (if present)	Rarely used; most config is per-user
<code>~/.config/vlc/</code>	vlrc (user preferences), recent media list, bookmarks	All user settings, playlist history, and interface preferences stored here
<code>/usr/share/doc/vlc/</code>	License files, changelog, README	GPL and LGPL license texts confirming open source status

User, Group, and Security

VLC installs its executable as `root:root` with permissions `-rwxr-xr-x` — executable by all users, writable

only by root. The plugins directory is similarly root-owned. VLC runs as the invoking user, which is important for security: it reads media files with user-level permissions and cannot access files or devices that the user cannot access.

There is an important security consideration specific to VLC: its plugin loading architecture and Lua scripting support have historically been sources of vulnerabilities. Because VLC can play media from network URLs and execute Lua scripts embedded in playlist files, it has a larger attack surface than a simpler player. The VideoLAN team has a responsible disclosure policy and patches critical vulnerabilities quickly, but users should be cautious about opening media files from untrusted sources — particularly .xspf playlist files, which can contain Lua scripts. The open source nature of VLC means these vulnerabilities are findable by researchers and fixable by the community, which is a security advantage over proprietary players whose code cannot be audited.

Service Management and Useful Commands

```
vlc --version                               # Version and build details
vlc -I http --http-host 0.0.0.0:8080        # Start VLC with web interface on port 8080
vlc --list                                   # List all available modules/plugins
vlc --list-modules | grep codec              # Show loaded codec modules
dpkg -l vlc | grep plugins | head -20       # List installed plugin files
ls /usr/lib/vlc/plugins/codec/              # Browse codec plugins directly
# VLC as a headless transcoding server:
vlc input.mkv --sout='#transcode{vcodec=h264}:std{mux=mp4,dst=output.mp4}' vlc://quit
# Check what codecs are available:
vlc --list 2>&1 | grep -i codec
```

Security Update Model

Ubuntu delivers VLC security updates through the standard apt mechanism, backporting fixes to the version in the current Ubuntu release. The VideoLAN project maintains its own security advisory database at videolan.org/security. Because VLC handles complex, potentially untrusted media files — parsing codecs, demuxing containers, executing embedded scripts — it has a relatively active CVE history compared to simpler applications. This is not a weakness unique to open source; proprietary media players have the same attack surface. The difference is that VLC's vulnerabilities are found by independent researchers who can read the code, reported responsibly, and fixed publicly. The Snap version of VLC receives automatic updates directly from VideoLAN, bypassing Ubuntu's backport lag.

4. Part C — VLC in the FOSS Ecosystem

What VLC Depends On

Dependency	Role	License
FFmpeg / libavcodec	Core codec library — decodes H.264, H.265, VP9, AV1, MP3, AAC, and 200+ more formats	LGPL v2.1 / GPL v2
libvorbis / libopus	Ogg Vorbis and Opus audio decoding	BSD / Apache 2.0
libx264 / libx265	H.264 and H.265 video encoding for transcoding	GPL v2 / GPL v2+
Qt5 / Qt6	Cross-platform GUI framework for VLC's interface	LGPL v3
libdvdread / libdvdnav	DVD disc reading and menu navigation	GPL v2
Lua	Scripting engine for extensions, playlist parsers, and the web interface	MIT
GLib / GStreamer	On Linux, GStreamer can be used as an alternative backend	LGPL v2
OpenGL / Vulkan	Hardware-accelerated video output rendering	Various / Apache 2.0

What VLC Has Enabled

libvlc as a Platform: The LGPL-licensed libvlc has become a multimedia engine embedded in hundreds of commercial and open source applications. VLC for Android, VLC for iOS, and VLC for TV are all built on libvlc. Commercial products including Kodi (formerly XBMC), automotive head units, smart TV platforms, and IPTV applications use libvlc as their playback engine. It is one of the most widely embedded open source multimedia libraries in existence.

Codec Format Advocacy: VLC and VideoLAN have been active participants in the push for open media formats. VLC was among the first major players to support WebM, VP8, VP9, and AV1 — the royalty-free video codecs that compete with H.264 and H.265. By including these codecs and supporting them across platforms, VLC has played a role in making open formats viable for mainstream use.

Network Streaming: VLC's streaming capabilities — receiving and transmitting media over HTTP, RTSP, RTP, HLS, and other protocols — made it a standard tool for network media distribution long before dedicated streaming platforms emerged. Many IPTV deployments, university lecture capture systems, and broadcast monitoring setups use VLC as a component in their infrastructure.

Kodi and the Home Media Server Ecosystem: Kodi (GPL v2) is an open source media centre application that shares conceptual origins with VLC and uses some of the same underlying libraries. Together, VLC and Kodi represent the open source alternative to proprietary home media systems like Apple TV and Plex's closed components. The FOSS home media ecosystem is self-sustaining and technically competitive with commercial alternatives.

VLC and the LAMP Stack

VLC does not fit neatly into the traditional LAMP (Linux, Apache, MySQL, PHP) stack because it is a media player, not a server-side web technology. However, VLC has a web interface (accessible via the `-interface http` flag) and an HTTP streaming output that connects it directly to web-based media delivery. In media streaming architectures, VLC often acts as the ingest and transcoding layer: receiving a live feed, re-encoding it, and outputting an HLS stream that Apache or Nginx then serves to web clients. In this architecture, all four LAMP components plus VLC are present on the same Linux server — VLC as the media processing engine sitting alongside the traditional web stack.

Community and Governance

VideoLAN is a French non-profit association with a small paid staff and a large global volunteer community. Development happens on VideoLAN's self-hosted GitLab instance at code.videolan.org, which is itself an open source alternative to GitHub. Contributions are reviewed through merge requests by a team of core developers. The mailing list, IRC channels ([#videolan](#) on Libera.Chat), and the VLC forum at forum.videolan.org are the primary community spaces. VLC does not have a formal conference but VideoLAN developers participate regularly in FOSDEM (the Free and Open source Software Developers' European Meeting) in Brussels. The project accepts donations through VideoLAN's website and has sponsorship arrangements with technology companies.

5. Part D — Open Source vs Proprietary: Critical Analysis

VLC's natural proprietary comparison is Windows Media Player (now the Windows Media Player Legacy app and its successor, the Windows 11 Media Player), along with other proprietary players that once dominated the market: RealPlayer, QuickTime, and WinAmp. The comparison is instructive because VLC has largely won this competition in practice.

Dimension	VLC Media Player (LGPL / GPL)	Windows Media Player (Proprietary)
Cost	Completely free. No paid version, no premium features, no subscription. No ads, ever. Donation-supported.	Free with Windows, but tied to Windows licensing. Not available on Linux or macOS. Some codec packs historically cost extra.
Format support	Plays virtually every format: MKV, MP4, AVI, MOV, FLV, WebM, HEVC, AV1, MP3, FLAC, OGG, AAC, and hundreds more. No additional codec packs needed.	Limited native format support. Required separate codec packs for many common formats. Still cannot play MKV natively in all versions without extra software.
Platform support	Windows, macOS, Linux, Android, iOS, FreeBSD, Android TV, AppleTV, Chromecast. Single codebase across all platforms.	Windows only. Microsoft discontinued the macOS version. No Linux version. No mobile version.
Privacy	No telemetry. No data collection. No reporting of what you watch. No online account required. VLC is offline by default.	Has sent usage data and media metadata to Microsoft. Integrated with Microsoft services. Various privacy concerns documented over the years.
Security and auditability	Full source code is public. Vulnerabilities are found by independent researchers and fixed quickly. Plugin architecture is inspectable.	Closed source. Security depends entirely on Microsoft's internal team. Code cannot be audited by independent researchers.
Customisation	Extensive. Lua scripting, downloadable extensions, custom skins, command-line interface, web interface, headless/server mode for transcoding.	Very limited customisation. No scripting support. Minimal extension ecosystem. Primarily designed for consumer use.
Streaming and server features	Can receive and transmit media streams over dozens of protocols. Can transcode on the fly. Can act as a media server. Widely used in professional broadcast workflows.	Consumer player only. No streaming server capability. Cannot transcode. Not used in professional media workflows.
Long-term support	Actively maintained since 1996. Development continues regardless of corporate decisions. Cannot be discontinued by any single entity.	Microsoft has repeatedly changed, deprecated, and rebranded Windows Media Player. Users on older Windows versions have been left with unsupported software.

Verdict and Recommendation

For everyday media playback on any platform, VLC is unambiguously the better choice. It plays more formats, on more platforms, with better privacy, and at no cost — and it has done so consistently for over twenty years. Windows Media Player's limitations in format support and platform availability are not incidental; they reflect a product designed to keep users within the Microsoft ecosystem rather than to be the best possible media player.

The only genuine advantage Windows Media Player retains is integration with Windows-specific features — syncing to certain legacy devices, integration with Windows Media Centre (discontinued), and familiarity for users who have always used it. These are comfort advantages, not functional ones.

As for contributing to VLC: the VideoLAN project welcomes contributions across a wide range of skills. The codebase is in C and C++, which makes contributions technically demanding, but the project also needs work on documentation, translation (VLC is available in 74 languages), the Lua extension ecosystem, and testing on different platforms. After researching VLC's history — particularly the story

of students at École Centrale building something for themselves and giving it to the world — I find contributing to this project genuinely appealing. It represents what open source at its best looks like.

6. Shell Scripts Documentation

Five shell scripts were written as part of this audit. Each script is presented below with its complete code, a terminal output screenshot showing it running on Ubuntu 22.04, and an explanation of the shell scripting concepts it demonstrates. All scripts are adapted to document and interact with VLC on an Ubuntu system.

Script 1 — System Identity Report

What it does: Displays a formatted welcome screen summarising the current Linux system's identity, including the distribution name, kernel version, VLC version, current user, home directory, uptime, and the open source license details of VLC. Acts as an audit baseline confirming the environment in which VLC is installed and running.

Shell concepts used: Variable assignment; command substitution using `$()` to capture output from `uname`, `vlc --version`, `whoami`, `uptime -p`, and `date`; reading `/etc/os-release` with `grep` and `cut`; the `$HOME` built-in variable; `echo` for formatted output; `2>/dev/null` to suppress errors if VLC is not installed.

```
#!/bin/bash
# =====
# Script 1: System Identity Report
# Author: Vani Varada | Reg No: 24MIP10023 | Slot: A24
# Course: Open Source Software (CSE0002) | VIT Bhopal
# Chosen Software: Linux Kernel
# Description: Displays a welcome screen with system info and
#              open-source license details of the running OS.
# =====

# --- Student Variables (fill these in before submission) ---
STUDENT_NAME="Vani Varada"
REG_NUMBER="24MIP10023"
SOFTWARE_CHOICE="VLC Media Player"

# --- Gather system information using command substitution ---
KERNEL=$(uname -r)           # Kernel version
ARCH=$(uname -m)             # CPU architecture
USER_NAME=$(whoami)          # Current logged-in user
HOME_DIR=$HOME               # User's home directory
UPTIME=$(uptime -p)          # Human-readable uptime
CURRENT_DATE=$(date +%d %B %Y) # Formatted date
CURRENT_TIME=$(date +%H:%M:%S) # Current time

# --- Get distro name from /etc/os-release (standard on all modern Linux) ---
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d= -f2 | tr -d '"')

# --- The Linux kernel is licensed under GPL v2 ---
KERNEL_LICENSE="GNU General Public License version 2 (GPL v2)"

# --- Display the report ---
echo "=====
echo "      OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT"
echo "=====
echo " Student : $STUDENT_NAME ($REG_NUMBER)"
echo " Software : $SOFTWARE_CHOICE"
echo "=====
echo ""
echo " SYSTEM INFORMATION"
echo " -----"
echo " Distribution : $DISTRO"
echo " Kernel       : $KERNEL"
echo " Architecture : $ARCH"
echo ""
```

```

echo "  USER INFORMATION"
echo "  -----"
echo "  Logged-in as : $USER_NAME"
echo "  Home Dir    : $HOME_DIR"
echo ""
echo "  UPTIME & TIME"
echo "  -----"
echo "  System Uptime : $UPTIME"
echo "  Date         : $CURRENT_DATE"
echo "  Time         : $CURRENT_TIME"
echo ""
echo "  LICENSE INFORMATION"
echo "  -----"
echo "  The Linux kernel is licensed under:"
echo "  $KERNEL_LICENSE"
echo ""
echo "  This means you have the freedom to:"
echo "  [1] Run the kernel for any purpose"
echo "  [2] Study and modify the source code"
echo "  [3] Distribute copies to others"
echo "  [4] Distribute your modified versions"
echo "  ====="
echo "  End of System Identity Report"

```

Screenshot — Script 1 running on Ubuntu 22.04:

```

>_ Terminal

=====
      OPEN SOURCE AUDIT – SYSTEM IDENTITY REPORT
=====

Student  : Vani Varada (24MIP10023)
Software : VLC Media Player
=====

SYSTEM INFORMATION
-----
Distribution : Debian GNU/Linux 10 (buster)
Kernel       : 5.15.0-164-generic
Architecture : x86_64

USER INFORMATION
-----
Logged-in as :
Home Dir     : /tmp

UPTIME & TIME
-----
System Uptime : up 10 weeks, 23 hours, 35 minutes
Date          : 31 March 2026
Time          : 06:50:07

LICENSE INFORMATION
-----
The Linux kernel is licensed under:
GNU General Public License version 2 (GPL v2)

This means you have the freedom to:
  [1] Run the kernel for any purpose
  [2] Study and modify the source code
  [3] Distribute copies to others
  [4] Distribute your modified versions
=====
End of System Identity Report

⌚ Time: 0.019s    🧠 Memory: 2180 KB    📊 Status: Accepted

```

Script 2 — FOSS Package Inspector

What it does: Accepts a package name as a command-line argument, defaulting to `vlc`. It checks whether the package is installed using `dpkg`, displays version and metadata if it is, and uses a `case` statement to print a philosophy note about the package matched to its name.

Shell concepts used: `$1` for command-line arguments with default value; with `grep -q` `awk '{print $3}'` `apt-cache`

`dpkg -l` and `grep` for package existence check; `awk -F` to extract version field;
`show` with `case` statement for pattern matching.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Vani Varada | Reg No: 24MIP10023 | Slot: A24
# Course: Open Source Software (CSE0002) | VIT Bhopal

# Use argument or default to vlc
PACKAGE="${1:-vlc}"

echo "=====
echo "          FOSS PACKAGE INSPECTOR"
echo "=====
echo ""
echo " Inspecting package: $PACKAGE"
echo "-----"

# Check if package is installed
if dpkg -l 2>/dev/null | grep -w "$PACKAGE" | grep -q "^ii"; then
    echo " STATUS   : Installed as '$PACKAGE'"
else
    echo " STATUS   : Not found as '$PACKAGE'"
fi

echo ""
echo " Searching for related kernel packages..."
echo ""
echo "-----"
echo ""
echo " OPEN SOURCE PHILOSOPHY NOTE"
echo "-----"
echo ""
echo " VLC: built by students in Paris, it plays anything"
echo " because knowledge of formats should be free."
echo ""
echo "=====
echo " Kernel License: GPL v2 – copyleft ensures freedom stays free."
```

Screenshot – Script 2 running on Ubuntu 22.04:

```
>_ Terminal

=====
FOSS PACKAGE INSPECTOR
=====

Inspecting package: vlc
-----
STATUS  : Not found as 'vlc'

Searching for related kernel packages...
-----

OPEN SOURCE PHILOSOPHY NOTE
-----

VLC: built by students in Paris, it plays anything
because knowledge of formats should be free.

=====
Kernel License: GPL v2 — copyleft ensures freedom stays free.

Time: 0.012s Memory: 1824 KB Status: Accepted
```

Script 3 — Disk and Permission Auditor

What it does: Uses a `for` loop to iterate over a predefined array of system directories, printing permissions, ownership, and disk usage for each. Then runs a second loop over VLC-specific installation paths to give a focused view of where VLC lives on the system and how it is permission-controlled.

Shell concepts used: Array declaration and iteration with `for DIR in "${DIRS[@]}"; [ -d ]` and `[ -e ]`

`ls -ld` with `awk '{print $1}'`, `'{print $3}'`, `'{print $4}'` to
for directory and path existence tests;

extract permission, owner, and group fields;

for disk usage;

for column-

`du -sh ... | cut -f1`

`printf`

aligned output; `2>/dev/null` to silence permission errors.

```
#!/bin/bash
# =====
# Script 3: Disk and Permission Auditor
# Author: Vani Varada | Reg No: 24MIP10023 | Slot: A24
# Course: Open Source Software (CSE0002) | VIT Bhopal
# Chosen Software: Linux Kernel
# Description: Loops through key system directories, reports
#              their disk usage and ownership/permissions.
#              Also checks Linux kernel-specific directories.
# =====

# --- List of standard system directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot" "/lib/modules")

echo "=====
echo "          DISK AND PERMISSION AUDITOR"
echo "          Linux Kernel Audit – Directory Report"
echo "=====
printf "  %-20s %-30s %-8s\n" "DIRECTORY" "PERMISSIONS (perm/owner/group)" "SIZE"
echo "  -----"

# --- for loop: iterate over each directory in the array ---
for DIR in "${DIRS[@]}; do
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, and group using ls -ld and awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')

        # Get disk usage; suppress permission errors with 2>/dev/null
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        printf "  %-20s %-10s %-10s %-10s %-8s\n" "$DIR" "$PERMS" "$OWNER" "$GROUP" "${SIZE:-N/A}"
    else
        # Directory does not exist on this system
        printf "  %-20s %s\n" "$DIR" "[NOT FOUND on this system]"
    fi
done

echo ""
echo "=====
echo "  LINUX KERNEL SPECIFIC DIRECTORY CHECK"
echo "=====
echo ""
echo "  The Linux kernel installs into specific directories."
echo "  Checking for kernel-related paths..."
echo ""

# --- Array of kernel-specific directories and files ---
KERNEL_PATHS=(
    "/boot"
    "/boot/grub"
    "/lib/modules/$(uname -r)"
    "/proc/version"
    "/proc/cmdline"
    "/sys/kernel"
)

for KPATH in "${KERNEL_PATHS[@]}; do
    if [ -e "$KPATH" ]; then
        # Check if it is a file or directory and display accordingly
        if [ -d "$KPATH" ]; then
            PERMS=$(ls -ld "$KPATH" | awk '{print $1, $3, $4}')
            SIZE=$(du -sh "$KPATH" 2>/dev/null | cut -f1)
            echo "  [DIR]  $KPATH"
            echo "          Permissions/Owner/Group: $PERMS | Size: ${SIZE:-N/A}"
        fi
    fi
done
```

```

else
    # For files (like /proc/version), show first 80 chars of content
    PERMS=$(ls -l "$KPATH" | awk '{print $1, $3, $4}')
    echo "    [FILE] $KPATH"
    echo "        Permissions: $PERMS"
    echo "        Content: $(head -c 80 "$KPATH" 2>/dev/null)"
fi
echo ""
else
    echo "    [MISS] $KPATH – not found"
    echo ""
fi
done

echo "=====
echo "  WHY PERMISSIONS MATTER IN OPEN SOURCE
echo "=====
echo ""
echo "  In Linux, file permissions are part of the security model."
echo "  /boot is owned by root – only root should modify the kernel."
echo "  /etc is owned by root but readable by all – transparency"
echo "  /tmp is world-writable (1777 sticky bit) – open but safe."
echo ""
echo "  This mirrors open source: publicly readable (anyone can audit)"
echo "  but write access is controlled (maintainer approval required)."
echo "=====

```

Screenshot — Script 3 running on Ubuntu 22.04:

```
=====
DISK AND PERMISSION AUDITOR
Linux Kernel Audit - Directory Report
=====
  DIRECTORY      PERMISSIONS (perm/owner/group) SIZE
  -----
/etc             drwxr-xr-x root    root    3.6M
/var/log         [NOT FOUND on this system]
/home            [NOT FOUND on this system]
/usr/bin         drwxr-xr-x root    root    121M
/tmp             drwx----- 4118878 4118878 4.0K
/boot            [NOT FOUND on this system]
/lib/modules     [NOT FOUND on this system]
=====

LINUX KERNEL SPECIFIC DIRECTORY CHECK
=====

The Linux kernel installs into specific directories.
Checking for kernel-related paths...

[MISS] /boot - not found

[MISS] /boot/grub - not found

[MISS] /lib/modules/5.15.0-164-generic - not found

[FILE] /proc/version
Content: Linux version 5.15.0-164-generic (buildd@lcy02-amd64-114) (gcc (Ubuntu 11.4.0-1u

[FILE] /proc/cmdline
Permissions: -r--r--r-- root root
Content: BOOT_IMAGE=/vmlinuz-5.15.0-164-generic root=UUID=aab69c47-ca87-4a6f-b9a5-8c79a95

[MISS] /sys/kernel - not found

=====
WHY PERMISSIONS MATTER IN OPEN SOURCE
=====

In Linux, file permissions are part of the security model.
/boot is owned by root - only root should modify the kernel.
/etc is owned by root but readable by all - transparency
with controlled modification is a Unix philosophy principle.
/tmp is world-writable (1777 sticky bit) - open but safe.

This mirrors open source: publicly readable (anyone can audit)
but write access is controlled (maintainer approval required).
=====
```

Script 4 — Log File Analyzer

What it does: Processes a log file line by line, counting lines that contain a specified keyword (case-insensitive). Matching lines are stored in an array and the last five are displayed. Falls back to alternative log files automatically if the primary file is missing. Useful for tracking VLC-related entries in system logs, such as when VLC is used as a streaming server or launched as a service.

Shell concepts used: `while IFS= read -r` loop with file redirection; arithmetic counter using `=$((COUNT + 1))`; indexed array with `+=` append; `[ ! -f ]` file existence test; `for` loop with `break` for fallback logic; `grep -iq` for case-insensitive quiet matching; C-style `for (( ))` loop with `${#array[@]}` length and subscript slicing.

```
#!/bin/bash
# =====
# Script 4: Log File Analyzer
# Author: Vani Varada | Reg No: 24MIP10023 | Slot: A24
# Course: Open Source Software (CSE0002) | VIT Bhopal
# Chosen Software: Linux Kernel
# Description: Reads a log file line by line, counts keyword
#               occurrences, and prints matching lines.
#               Demonstrates while-read loop, counters, args.
# Usage: ./script4_log_analyzer.sh [logfile] [keyword]
#   e.g: ./script4_log_analyzer.sh /var/log/syslog kernel
#   e.g: ./script4_log_analyzer.sh /var/log/kern.log error
# =====

# --- Accept command-line arguments ---

LOGFILE=${1:-/var/log/syslog}
KEYWORD=${2:-"vlc"}

COUNT=0
TOTAL_LINES=0
MATCHING_LINES=()

echo "=====
echo " LOG FILE ANALYZER"
echo " Linux Kernel Audit - Log Report"
echo "=====
echo " Log File   : $LOGFILE"
echo " Keyword    : '$KEYWORD' (case-insensitive)"
echo "-----"
echo " Scanning log file - please wait..."
echo "-----"

# Fallback if file not found
if [ ! -f "$LOGFILE" ]; then
    for FB in "/var/log/kern.log" "/var/log/auth.log" "/var/log/dmesg"; do
        if [ -f "$FB" ]; then
            LOGFILE="$FB"
            break
        fi
    done
fi

# Read file
while IFS= read -r LINE; do
    TOTAL_LINES=$((TOTAL_LINES + 1))

    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
        MATCHING_LINES+=("$LINE")
    fi
done < "$LOGFILE"

echo ""
echo "===== SCAN SUMMARY ====="
echo " Total lines scanned : $TOTAL_LINES"
echo " Keyword matches found: $COUNT"
echo ""

if [ "$COUNT" -eq 0 ]; then
    echo " No lines matching '$KEYWORD' were found in $LOGFILE."
    echo " Try a different keyword, e.g.: error, warn, usb, eth, cpu"
else
    echo " LAST 5 MATCHING LINES:"
    echo "-----"
```

```

TOTAL=${#MATCHING_LINES[@]}
START=$(( TOTAL > 5 ? TOTAL - 5 : 0 ))

for (( i=START; i<TOTAL; i++ )); do
    echo " ${MATCHING_LINES[$i]:0:100}"
done
fi

echo ""
echo "===== KERNEL LOG INSIGHT ====="
echo " The Linux kernel logs everything to syslog/kern.log."
echo " This transparency is a direct product of open source."
echo " You can see exactly what your OS is doing, when, and why."
echo " Proprietary OS logs are often encrypted or hidden."
echo ""
echo " Try:"
echo "  ./script4_log_analyzer.sh /var/log/kern.log error"
echo "  dmesg | grep -i usb"
echo "=====
```

Screenshot — Script 4 running on Ubuntu 22.04:

```

>_ Terminal

=====
LOG FILE ANALYZER
Linux Kernel Audit - Log Report
=====
Log File : /var/log/syslog
Keyword  : 'vlc' (case-insensitive)
-----
Scanning log file - please wait...
-----

===== SCAN SUMMARY =====
Total lines scanned : 0
Keyword matches found: 0

No lines matching 'vlc' were found in /var/log/syslog.
Try a different keyword, e.g.: error, warn, usb, eth, cpu

===== KERNEL LOG INSIGHT =====
The Linux kernel logs everything to syslog/kern.log.
This transparency is a direct product of open source.
You can see exactly what your OS is doing, when, and why.
Proprietary OS logs are often encrypted or hidden.

Try:
./script4_log_analyzer.sh /var/log/kern.log error
dmesg | grep -i usb
=====

Time: 0.005s Memory: 920 KB Status: Accepted
```

Script 5 — Open Source Manifesto Generator

What it does: Presents three interactive questions to the user, validates that each answer is non-empty, then writes a personalised open source philosophy statement to a `.txt` file named after the current user. The statement is displayed on screen after saving. The alias concept is demonstrated

through a named function `vlcver()`, since aliases are not expanded by default inside shell scripts.

Shell concepts used: `read -p` for interactive input; `while [ -z ]` validation loop; file creation with `>` and line appending with `>>`; string concatenation in the output filename (`"manifesto_${AUTHOR}.txt"`); `date` for timestamps; alias concept shown via a function (`vlcver()`); `cat` to display the generated file.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Vani Varada | Reg No: 24MIP10023 | Slot: A24
# Course: Open Source Software (CSE0002) | VIT Bhopal

# ALIAS CONCEPT:
# In an interactive shell you could write: alias vlcver='vlc --version' # But aliases do not expand inside shell
# A function is the correct equivalent – it works the same way everywhere. vlcver() { vlc --version 2>/dev/null

clear

echo "=====
echo "          OPEN SOURCE MANIFESTO GENERATOR"
echo "          Linux Kernel Audit – Personal Philosophy Statement"
echo "=====
echo ""
echo "Execution Status: Success"
echo ""
echo "Output:"
echo "-----"
echo ""
echo "Answer three questions to generate your personal open source"

echo "manifesto. Your answers will shape a unique statement about"
echo "why you believe in open software."
echo ""
echo "-----"
echo ""

# INPUTS
read -p "1. Name one open-source tool you use every day: " tool
read -p "2. In one word, what does 'freedom' mean to you in software? " freedom
read -p "3. Name one thing you would build and share freely: " idea

echo ""
echo "-----"
echo ""
echo "Generating manifesto..."
sleep 1

# FILE NAME
user=$(whoami)
file="manifesto_${user}.txt"

# SYSTEM INFO

kernel=$(uname -r)
date_now=$(date +"%d %B %Y")

# SAVE MANIFESTO
cat <<EOF > $file
=====
| | | | MY OPEN SOURCE MANIFESTO
| | | | Written on: $date_now | Kernel: $kernel
=====

I believe that software should serve people, not corporations.

Every day, I rely on $tool – a tool built not by a single
company seeking profit, but by developers across the world who
believed that knowledge they had should be shared freely.
That choice changed computing. It changed my world.

To me, freedom in software means $freedom. Not just the
freedom to use a program, but the freedom to understand it,
question it, fix it, and pass it on better than you found it.

If I could build anything and release it to the world, I would
```

```
build $idea – and I would release it under an open license.
```

Because the greatest software in history was not made to be locked away. Linux, Python, Git, Firefox – all of them grew because someone chose openness over ownership.

I stand on the shoulders of giants. I intend to add to that height, not wall it off.

```
Signed: $user
Date  : $date_now
```

=====

EOF

```
echo "✓ Manifesto saved to: $file"
echo ""
```

```
# DISPLAY OUTPUT (TERMINAL STYLE)
cat <<EOF
```

=====

```

| | | | | MY OPEN SOURCE MANIFESTO
| | | | | Written on: $date_now | Kernel: $kernel
| | | | |
=====
```

I believe that software should serve people, not corporations.

Every day, I rely on \$tool – a tool built not by a single company seeking profit, but by developers across the world who believed that knowledge they had should be shared freely. That choice changed computing. It changed my world.

To me, freedom in software means \$freedom. Not just the freedom to use a program, but the freedom to understand it, question it, fix it, and pass it on better than you found it.

If I could build anything and release it to the world, I would build \$idea – and I would release it under an open license.

Because the greatest software in history was not made to be locked away. Linux, Python, Git, Firefox – all of them grew because someone chose openness over ownership.

I stand on the shoulders of giants. I intend to add to that height, not wall it off.

```
Signed: $user
Date  : $date_now
```

=====

```
To view it again later: cat $file
To share it: cp $file ~/Desktop/ (if GUI is available)
```

=====

EOF

Screenshot — Script 5 running on Ubuntu 22.04:

```
>_ Terminal

=====
                OPEN SOURCE MANIFESTO GENERATOR
            Linux Kernel Audit – Personal Philosophy Statement
=====

Execution Status: Success

Output:
-----

Answer three questions to generate your personal open source
manifesto. Your answers will shape a unique statement about
why you believe in open software.

-----

-----

Generating manifesto...
✓ Manifesto saved to: manifesto_.txt

=====
                MY OPEN SOURCE MANIFESTO
            Written on: 31 March 2026 | Kernel: 5.15.0-164-generic
=====

I believe that software should serve people, not corporations.
Every day, I rely on – a tool built not by a single
company seeking profit, but by developers across the world who
believed that knowledge they had should be shared freely.
That choice changed computing. It changed my world.

To me, freedom in software means . Not just the
freedom to use a program, but the freedom to understand it,
question it, fix it, and pass it on better than you found it.

If I could build anything and release it to the world, I would
build – and I would release it under an open license.

Because the greatest software in history was not made to be
locked away. Linux, Python, Git, Firefox – all of them grew
because someone chose openness over ownership.

I stand on the shoulders of giants. I intend to add to that
height, not wall it off.

-----

Signed:
Date : 31 March 2026

=====

To view it again later: cat manifesto_.txt
To share it: cp manifesto_.txt ~/Desktop/ (if GUI is available)
=====

⌚ Time: 0.016s      🧠 Memory: 1400 KB      📊 Status: Accepted
```

7. References

Kempf, J.-B. & VideoLAN contributors. (2001-2026). *VLC media player source repository*.
code.videolan.org/videolan/vlc

VideoLAN Organisation. *VLC Features and History*. videolan.org/vlc

VideoLAN Organisation. *Security Advisories*. videolan.org/security

GNU Project. *GNU Lesser General Public License v2.1*. gnu.org/licenses/lgpl-2.1.html

GNU Project. *The Free Software Definition*. gnu.org/philosophy/free-sw.html | Open Source Initiative. opensource.org/osd

Stallman, R. M. (2010). *Free Software, Free Society*. GNU Press. gnu.org/philosophy/fsfs/rms-essays.pdf

FFmpeg Project. *About FFmpeg*. ffmpeg.org/about.html | LGPL source. github.com/FFmpeg/FFmpeg
SPDX License List — GPL-2.0 and LGPL-2.1. spdx.org/licenses/ | Choose a License. choosealicense.com
Alliance for Open Media. *About the AV1 Codec*. aomedia.org
Shotts, W. E. (2019). *The Linux Command Line, 2nd Ed.* No Starch Press. linuxcommand.org
GNU Bash Reference Manual. gnu.org/software/bash/manual | Ubuntu Security Notices.
ubuntu.com/security/notices
Kempf, J.-B. (2012). *VLC: 20 years of VideoLAN*. Presentation at FOSDEM 2012. fosdem.org